

DEVOPS INTERVIEW PREPARATION GUIDE

Sanket Ajay Chopade — Fresher DevOps Engineer
Follow-Up Questions & Cross-Questions with Model Answers

Based on resume: Hisan Labs Internship | ApnaKart | Flight Reservation System

PART 1 — FOLLOW-UP QUESTIONS

CI/CD & Jenkins

Q1: You mentioned reducing deployment time by 60% using Jenkins. How did you actually achieve that reduction?

Before the Jenkins pipeline, deployments were mostly manual — developers would build locally, push images manually, and SSH into servers to update containers. We automated all of that into a single pipeline: GitHub webhook triggers the build on every merge to main, Maven compiles and runs tests, SonarQube checks code quality, Trivy scans the Docker image for vulnerabilities, and then the image is pushed to ECR and deployed to EKS using a rolling update strategy. Eliminating manual steps and running stages in automated sequence is what drove that 60% reduction.

Q2: What happens if SonarQube or Trivy fails in your pipeline? Does the deployment stop?

Yes. We configured quality gates in SonarQube — if code coverage drops below threshold or critical issues are found, the pipeline fails and deployment doesn't proceed. Same with Trivy — we set severity thresholds so CRITICAL vulnerabilities block the pipeline. This is the DevSecOps approach: shift security left so issues are caught before they ever reach Kubernetes.

AWS & Terraform

Q3: You used Terraform with remote state on S3 + DynamoDB. Why is that important and what problem does it solve?

If you store Terraform state locally, two engineers running 'terraform apply' simultaneously can corrupt the state file or overwrite each other's changes — causing infrastructure drift. Remote state on S3 keeps a single source of truth, and DynamoDB provides state locking, so only one operation can run at a time. It also means the CI/CD pipeline itself can run Terraform safely without any developer's local machine being involved.

Q4: How did you manage secrets in Terraform? Were you storing credentials in .tfvars files?

No — we avoided hardcoding credentials. For AWS authentication, we used IAM roles attached to the Jenkins agent itself, so no access keys were stored in code. For sensitive values like DB passwords, we either passed them as environment variables during CI runs or referenced AWS Secrets Manager. The .tfvars files with non-sensitive config were committed, but anything sensitive was excluded via

.gitignore and injected at runtime.

Docker & Kubernetes

Q5: You reduced your Docker image from 450MB to 180MB. Walk me through exactly how you did that.

The key was a multi-stage Dockerfile. Stage 1 used `maven:3.9-temurin-17` as the builder — large because it contains Maven, the JDK, source code, and all build tools. Stage 2 used `eclipse-temurin:17-jre-alpine` as the final runtime — a minimal Alpine-based JRE, not the full JDK. I only copy the compiled `.jar` from Stage 1 into Stage 2. The final image has no source code, no Maven, no build tools — just the JRE and the artifact. That's what cut it from ~450MB to ~180MB.

Q6: What is HPA and how did you configure it in your project?

HPA stands for Horizontal Pod Autoscaler. It automatically scales the number of pod replicas based on observed metrics — we used CPU and memory utilization. In our Kubernetes manifests, we defined `minReplicas`, `maxReplicas`, and target CPU utilization percentage. For example, if CPU crosses 70%, Kubernetes spins up additional pods automatically. This was critical for handling traffic spikes without manual intervention and contributed to our 99%+ uptime.

Q7: What is the difference between a liveness probe and a readiness probe?

A liveness probe tells Kubernetes whether a container is alive — if it fails, Kubernetes restarts the container. A readiness probe tells Kubernetes whether the container is ready to receive traffic — if it fails, the pod is removed from the Service's load balancer endpoints but not restarted. We used readiness probes to prevent traffic from hitting a pod still initializing its Spring Boot context, and liveness probes to auto-recover pods stuck in a deadlock or crash loop.

DevSecOps & Security

Q8: You mentioned Kubernetes RBAC. Can you give a specific example of how you applied it?

We created ServiceAccounts for different components and bound them to Roles with only the permissions they needed. For example, application pods had a ServiceAccount that could only read ConfigMaps and Secrets within their namespace — they couldn't create or delete resources. Jenkins had a ServiceAccount with permission to apply deployments in specific namespaces but nothing else. This follows least-privilege — if a pod is compromised, the blast radius is contained.

Monitoring & Observability

Q9: You participated in RCA sessions with Datadog. Walk me through one incident you remember.

One incident involved a payment microservice throwing 500 errors intermittently. We noticed it in Datadog's APM — the service error rate spiked and latency increased. Drilling into traces, we saw the DB connection pool was getting exhausted under load. The RCA pointed to the connection pool size being too small relative to HPA scaling up more pods — each pod was competing for DB connections. The fix was two-part: increase the RDS connection limit and tune the Spring Boot connection pool settings per replica. We documented this in the runbook.

Architecture & Design

Q10: Your ApnaKart project uses the database-per-service pattern. Why not just use one shared database?

A shared database creates tight coupling — if one service changes its schema, it can break another. It also becomes a single point of failure and a performance bottleneck. With database-per-service, each service owns its data and can scale, fail, or be updated independently. The tradeoff is managing distributed data consistency, but for an e-commerce platform where Order, Payment, and Inventory need to evolve separately, the isolation is worth it.

Behavioral / Mindset

Q11: You're a fresher with internship experience. Why should we hire you over someone with 2-3 years of experience?

Fair question. I don't have the years — but I have real, production exposure to the full DevOps lifecycle, not just theory. I've built CI/CD pipelines that actually reduced deployment time, managed Kubernetes clusters with real uptime requirements, and dealt with incidents and RCAs. I'm also at a stage where I learn fast and have no legacy habits to unlearn. I'll absorb your team's practices quickly, contribute meaningfully from day one on tooling and automation, and I'm highly motivated because this is exactly the career I've chosen and prepared for.

PART 2 — CROSS-QUESTIONS (Verify You Lived Your Resume)

CI/CD — Prove It

Q1: You say you reduced deployment time by 60%. What was the deployment time BEFORE and AFTER? Give me the actual numbers.

Before the automated pipeline, a full deployment — building locally, manual Docker push, SSH into the server, pulling the image, restarting containers — used to take roughly 25-30 minutes end to end. After the Jenkins pipeline, a typical deployment from code push to pods running on EKS takes around 10-12 minutes. The 60% figure comes from that comparison. The biggest time savings came from eliminating manual steps and having all stages run automatically without human waiting time.

Q2: You listed Jenkins AND GitHub Actions AND GitLab CI. That's three CI/CD tools. Which one did you actually use in production?

Jenkins was our primary CI/CD tool at Hisan Labs for the ApnaKart project — that's where I have real production experience. GitHub Actions I used for the Flight Reservation System for simpler automation since it integrates natively with GitHub without needing a separate server. GitLab CI I worked with during my learning phase in a personal lab environment. Jenkins is where I have the deepest hands-on experience, but I understand how all three work conceptually.

Terraform — Deep Dive

Q3: You said Terraform ensures 'drift-free infrastructure.' But what if someone manually changes a security group in the AWS console? How does Terraform handle that?

That's exactly what infrastructure drift is. If someone manually changes a security group in the console, Terraform won't automatically detect it in real time — it only knows when you run 'terraform plan'. When you do run it, it compares the current state file with actual AWS state and shows the drift as a change. Running 'terraform apply' would then revert the manual change back to what's defined in code. To prevent this, we enforced a policy — all infrastructure changes must go through Terraform only, never the console. We also had periodic 'terraform plan' runs to catch drift early.

Q4: You used modular Terraform. Show me mentally what your module structure looked like.

We had a root main.tf that called child modules. The modules were: a 'vpc' module for VPC, subnets, route tables, and NAT gateway; an 'eks' module for the cluster, node groups, and IAM roles; an 'rds' module for the database instance, subnet groups, and security groups; and an 'iam' module for policies and role attachments. Each module had its own variables.tf, outputs.tf, and main.tf. The root module passed variables into child modules and consumed their outputs — for example, the EKS module consumed subnet IDs output from the VPC module.

Kubernetes — Deep Dive

Q5: You wrote manifests for 10 microservices across 4 environments. That's 40 sets of manifests. How did you manage that without going insane?

That's exactly why Kustomize was in our stack. We had a 'base' directory with core manifests for each service — Deployment, Service, ConfigMap, Ingress. Then 'overlays' for each environment — dev, test, uat, prod. Overlays only contained what changed per environment: image tags, replica counts, resource limits, environment-specific ConfigMap values. So we weren't maintaining 40 separate full manifest sets — we maintained 10 base sets and 4 lightweight overlay patches per service. Kustomize merges them at deploy time.

Q6: You mentioned rolling deployments. What is the risk with rolling deployments and how did you handle it?

The main risk is running two versions simultaneously during the rollout window — old pods and new pods serving traffic at the same time. If there's a breaking API change or DB schema migration, this can cause errors. We handled it by enforcing backward-compatible API changes — new endpoints were added, old ones deprecated but not removed immediately. For DB migrations, we used an expand-and-contract pattern: first add the new column, support both old and new schema in code, then once all pods are updated, run a second migration to remove the old column.

Q7: You listed resource requests and limits in Kubernetes. What's the difference, and what happens if you don't set them?

Requests are what Kubernetes uses for scheduling — it guarantees a pod will get at least that much CPU/memory on the node. Limits are the hard ceiling — if a pod exceeds its memory limit, it gets OOMKilled; if it exceeds CPU limit, it gets throttled. If you don't set them, the scheduler has no idea how to place pods efficiently, and a single misbehaving pod can consume all node resources and starve other pods — the noisy neighbor problem. We always set both for every microservice in production.

Security — Gotcha Questions

Q8: You said you used Kubernetes Secrets for sensitive data. But Kubernetes Secrets are just base64 encoded — that's not encryption. How is that actually secure?

You're right, and that's an important distinction. Base64 is encoding, not encryption — anyone with `kubectl` access can decode a Secret trivially. The real security comes from layering: we restricted who could access Secrets using RBAC — only specific ServiceAccounts that needed a Secret could read it. We also enabled encryption at rest in etcd using AWS KMS. We never mounted Secrets as environment variables in logs. The ideal next step would be integrating AWS Secrets Manager with the External Secrets Operator, which I've researched and documented as a future improvement.

Datadog / Observability

Q9: You configured Datadog APM. What's the difference between a metric, a trace, and a log? Why do you need all three?

They serve different purposes — this is the three pillars of observability. Metrics are numerical aggregates over time — like CPU usage, request rate, error rate. They tell you that something is wrong. Traces follow a single request across multiple services — they show you where the latency or error is happening. Logs give you the raw event detail — the exact error message, stack trace, request payload. In practice: a Datadog alert on error rate (metric) tells me something broke. APM traces show which service is failing. Then logs for that service show the exact exception. You need all three because metrics alone can't tell you why, and logs alone are too noisy to surface patterns.

Project — Ownership & Honesty

Q10: Your resume says ApnaKart was 'for Client Nova Technologies.' What was your actual individual contribution vs. the team's?

That's a fair and important distinction. ApnaKart was a team project in a 12-member Agile setup. My individual contributions were specifically in the DevOps/infrastructure domain: I personally wrote the Terraform modules for EKS and VPC, authored Kubernetes manifests for several microservices, configured the branch-to-namespace pipeline strategy in Jenkins, and set up Datadog dashboards and alerting. I didn't build the Spring Boot microservices or React frontend. I've been careful to use words like 'contributed to' and 'supported' on my resume rather than claiming sole ownership — that's the honest representation.

Q11: Your Flight Reservation System reduced provisioning time from 2 hours to 8 minutes. That's a very specific number. How did you arrive at it?

The 2-hour estimate for manual provisioning is based on what it realistically takes to manually set up an EKS cluster through the AWS console — creating the cluster, configuring node groups, setting up IAM roles, creating the VPC with subnets, then separately creating the RDS instance with the right security groups and subnet groups. If you've done it manually, 2 hours is actually conservative. The 8 minutes is the actual measured time for 'terraform apply' to run all those modules — I timed it during testing runs.

The 'Gotcha' Questions — Skills Depth

Q12: You have Azure and GCP listed in your skills. What have you actually done on Azure or GCP?

Honest answer — my hands-on depth is on AWS, which is where I have real project experience. For Azure and GCP, I've done labs and explored core concepts — Azure has AKS equivalent to EKS, Azure DevOps Pipelines equivalent to Jenkins, and I understand the conceptual mapping. GCP I've looked at GKE and Cloud Run. I listed them because I understand the ecosystem and can get productive quickly, but I won't pretend I've deployed production workloads there. If the role requires deep Azure or GCP work, I'd need a ramp-up period, and I'm transparent about that.

Q13: You listed Lambda in your AWS skills but there's no mention of it in any project. Have you actually used Lambda?

Good catch — Lambda doesn't appear in my project work because we didn't use it in the main projects. I've worked with Lambda in personal experiments — small event-driven functions triggered by S3 uploads — and I've read about it extensively in the context of serverless architectures. But I wouldn't claim deep production Lambda experience. If it came up in a role, I'm confident I could pick it up quickly given my solid AWS foundation, but I appreciate you pressing on that.

PRO TIP

Honest, nuanced answers build MORE interviewer trust than overconfident ones. Saying 'I haven't used Lambda in production but here's what I know and how I'd ramp up' is always stronger than bluffing. Interviewers reward self-awareness.